

1	2	3	4	Nota

Examen simulacro – 2do. cuatrimestre de 2026

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas.
- Numere las hojas entregadas. Complete en la primera hoja la cantidad total de hojas entregadas.
- Entregue esta hoja junto al examen, la misma **no** se incluye en la cantidad total de hojas entregadas.
- **Cada código o pseudocódigo debe estar bien explicado y justificado. ¡Obligatorio!**
- Toda suposición o decisión que tome deberá justificarla adecuadamente. Si la misma no es correcta o no se condice con el enunciado no será tomada como válida y será corregida acorde.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, antes de retirar el examen corregido del aula.
- Los parciales tienen dos notas: I (Insuficiente) y A (Aprobado).
- Para tener la nota aprobada es necesario tener al menos dos ejercicios calificados como bien y el restante al menos regular. Además, alguno de los ejercicios calificados como bien debe ser el de monitores o el de semáforos.

Ejercicio 1.

Considere la siguiente propuesta para resolver el problema de exclusión mutua para N threads (con N fijo), donde todos los threads tienen un id distinto tal que $0 \leq id < N$:

```
global bool[N] esperando = [false,...,false];
global int cantEsperando = 0;
global int proximo = -1;
```

```
thread (id) {
    //Seccion No Critica
    bool soyprimero = anotarse(id);
    if (soyprimero) llamarProximo();
    while (proximo!=id);
    //Seccion Critica
    bool soyultimo = desanotarse(id);
    if (!soyultimo) llamarProximo();
    //Seccion No Critica
}
```

Dadas las siguientes funciones.

```
bool anotarse(int idThread) {
    esperando[idThread]=true;
    cantEsperando++;
    return (cantEsperando==1);
}
```

```
bool desanotarse(int idThread) {
    esperando[idThread]=false;
    cantEsperando--;
    return (cantEsperando==0);
}
```

```
void llamarProximo() {
    int res = -1;
    for i in {N-1,...,0} {
        if (esperando[i]) res = i;
    }
    proximo = res;
}
```

Responder a las siguientes preguntas:

- a) Si se considera que `anotarse()`, `desanotarse()` y `llamarProximo()` no son atómicas, muestre adecuadamente que esta propuesta *no* resuelve el problema de la exclusión mutua.
- b) Considerando que `anotarse()`, `desanotarse()` y `llamarProximo()` son atómicas, ¿Resuelve esta propuesta el problema de la exclusión mutua? Justifique apropiadamente.
- c) ¿Alcanza con que `anotarse()`, `desanotarse()` y `llamarProximo()` sean atómicas para que el análisis del inciso b) siga valiendo al ejecutar este algoritmo en una arquitectura real, donde corre Java?

Para cada propiedad que en el inciso b) haya dado por válida, indique si el algoritmo se mantendría igual o si haría falta agregar restricciones para sostener esa garantía, y en ese caso cuál sería la restricción mínima que agregaría a nivel lenguaje y a nivel arquitectónico.

Para cada propiedad que en el inciso b) haya dado por no válida, indique si en este nuevo escenario sigue fallando por el mismo motivo, si aparecen motivos adicionales para que falle, o si las garantías de la arquitectura alcanzarían para cambiar esa conclusión.

Ejercicio 2.

Se desea modelar el procesamiento de transferencias en un banco con N cuentas. Cada transferencia se ejecuta mediante una operación `transferir(origen, destino, monto)`, con `monto` positivo, que mueve dinero de la cuenta `origen` a la cuenta `destino` (una transferencia nunca tiene la misma cuenta como origen y destino). El banco deja que las cuentas queden en rojo, pero solo hasta cierto punto: una transferencia se realiza únicamente si la cuenta de origen no queda con un saldo menor que $-L$, donde L es una constante conocida e igual para todas las cuentas. Si no es así, la transferencia se descarta.

Mientras una transferencia se está procesando, ningún otro proceso del banco puede ver ni modificar el saldo de ninguna de las dos cuentas involucradas. Los pedidos de transferencia llegan continuamente y de forma independiente (modele cada uno como un thread), sin ningún control centralizado sobre cuántos llegan a la vez, entre qué cuentas, ni en qué orden. Dos transferencias que no compartan ninguna cuenta deben poder procesarse simultáneamente.

Además, cada tanto el banco necesita saber cuántas cuentas quedaron en rojo, es decir con saldo negativo, y para eso cuenta con una operación `cuentasEnRojo()`. El resultado tiene que corresponder a un estado real del banco, sin transferencias a medio procesar. Como es un informe interno y nadie lo está esperando, se lo corre de madrugada, que es cuando el movimiento afloja: transferencias siguen llegando, pero hay ratos en los que no queda ninguna dando vueltas. Al banco no le molesta que el informe tenga que esperar su momento para arrancar. Lo que no tolera es lo contrario: que un informe que todavía no empezó a contar le haga esperar a una transferencia.

- a) Proponga, en pseudocódigo, una solución usando semáforos que resuelva el problema. Puede usar el constructor `Semaphore(n)` y las primitivas `acquire()` y `release()`. Salvo que se indique lo contrario, los semáforos se asumen débiles. La solución no debe tener deadlocks ni race conditions. Justifique todas las decisiones tomadas.
- b) Al banco le interesa que cada transferencia se procese lo antes posible, y le molesta que un pedido quede esperando mientras otros que llegaron después pasan adelante. ¿Puede ocurrir eso en su solución? ¿Hay algún límite para la cantidad de veces que un mismo pedido puede ser sobrepasado? Justifique su respuesta y, si corresponde, describa una traza.
- c) Suponga ahora que una transferencia puede mover dinero de una cuenta a varias a la vez, mediante una operación `transferirMultiple(origen, destinos, montos)`, donde todas las cuentas involucradas son distintas y la transferencia se descarta si el origen no alcanza para el total. Explique cómo modificaría su solución y si su justificación de ausencia de deadlock sigue siendo válida. No se pide reescribir la solución completa.

Ejercicio 3.

Inspirada en las noches de juegos de la facultad, Victoria decidió organizar algo similar en su casa para sus múltiples grupos de amigos, que en el fondo se detestan entre sí pero ninguno se lo pierde. Para eso cuenta con M mesas, donde la mesa i tiene `lugares[i]` asientos. A lo largo de la noche llegan grupos de amigos de distintos tamaños para jugar. Un grupo de k amigos necesita una mesa con al menos k asientos. El grupo no se separa y ocupa la mesa por completo aunque le sobren lugares, solo para no tener que compartirla con gente de otro grupo. Si un grupo al llegar no encuentra ninguna mesa libre, tienen que esperar hasta que se desocupe una, mirando con bastante rencor a los que la están ocupando.

Como Victoria sabe de estos problemas, nos pidió que simuláramos una posible solución. Lo único que nos pide es que, tarde o temprano, todos sus grupos de amigos consigan sentarse a jugar.

Modele esto mediante dos operaciones, `sentarse(k)` y `levantarse(mesa)`. Cada grupo se modela con un thread que, al llegar, invoca `sentarse(k)`, que le indica la mesa que le tocó y no retorna hasta conseguirla. Luego el grupo juega (esto no hace falta modelarlo) y, cuando termina, invoca `levantarse(mesa)` para liberarla.

Puede asumir que para todo grupo existe al menos una mesa en la que entra, y que todo grupo que se sienta se va en algún momento.

- Resuelva, en pseudocódigo, usando monitores con semántica de Mesa. Las colas de condición no son fair y no hay spurious wakeups. La solución no debe tener deadlocks ni race conditions. Justifique todas las decisiones tomadas.
- Explique en palabras cómo cambiaría su solución si el monitor tuviera semántica de Hoare. No se pide el código.

Ejercicio 4.

Se requiere diseñar un contador concurrente optimizado para alta escritura y baja lectura. Al crearse, la estructura inicializa un arreglo de N variables enteras para distribuir el nivel de concurrencia. Las operaciones de incremento se distribuirán entre estas variables para minimizar la contención. Por otro lado, la operación de lectura consolidará el total sumando el valor de las N variables. A continuación se muestra una implementación secuencial.

```
class Contador {  
  
    private int[] valor = new int[n];  
    private int tamano;  
  
    Contador(int n) {  
        tamano = n;  
        for (int i = 0; i < n; i++) {  
            valor[i] = 0;  
        }  
    }  
  
    public void inc(int id) {  
        valor[id % tamano]++;  
    }  
  
    public int get() {  
        int total = 0;  
        for (int i = 0; i < tamano; i++) {  
            total = total + valor[i];  
        }  
        return total;  
    }  
}
```

- Dar una solución de granularidad fina que permita la invocación concurrente de un incremento y de obtener el valor actual. Justificar la correctitud de su solución (en términos de linearizabilidad).
- Analizar el progreso del programa (presencia/ausencia de deadlocks, presencia/ausencia de starvation).
- Proponer una solución alternativa utilizando el enfoque de concurrencia *lock-free* (basada en operaciones CAS) para la operación de incremento. (**Nota:** Para este punto, no es necesario preocuparse por la implementación de `get()`).
- Considere que la operación `get()` se redefine mediante el siguiente algoritmo

```
public int get() {  
    while (true) {
```

```
    int total1 = 0;

    for (int i = 0; i < tamano; i++) {
        total1 = total1 + valor[i];
    }
    int total2 = 0;

    for (int i = 0; i < tamano; i++) {
        total2 = total2 + valor[i];
    }

    if (total1 == total2) {
        return total1;
    }
}
```

¿Es esta solución correcta (linearizable)? Justifique.

e) Indicar si en su solución el método `get()` es *wait-free*. Justificar.

Inciso extra (por fuera del parcial):

- Analice la correctitud (linearizabilidad) de una solución híbrida donde la operación `get()` mantiene la estrategia de granularidad fina definida en el Punto 1 (utilizando locks), pero las otras operaciones son las que utilizan CAS, con los incrementos optimistas.